

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CASE STUDY REPORT

RAILWAY RESERVATION AND PASSENGER SERVICE MANAGEMENT SYSTEM

Student Name:

Register Number:

Course Code: BCSE302P

Course Name: Database Systems Lab

Faculty Name:

Date of Submission:

1. Introduction

Indian Railways handles millions of reservations every day. A national railway organisation that still keeps reservation records in manual registers and spreadsheets faces double-booked seats, inconsistent fares, slow enquiries and no reliable picture of how full its trains are. This case study designs and implements a **Railway Reservation and Passenger Service Management System** as a relational database in **Oracle Database (SQL and PL/SQL)**.

The system stores the railway network (stations, routes and the ordered stops on each route), the rolling stock (trains, coaches, travel classes and every seat or berth), the dated runs of each train, registered passengers, and the complete ticket life-cycle: reservation, seat allocation, waitlist, payment, cancellation and refund. Business rules such as "a seat can never be sold twice on the same journey" and "the fare is always calculated from distance and class" are enforced inside the database by constraints, unique indexes and a trigger, so they cannot be bypassed by any application.

On top of the operational data, the system provides analytical reports for railway management (train-wise statistics, station traffic, route and coach utilisation, cancellations, revenue, passenger frequency) and a **Train Capacity Risk Identification Module** that classifies every train as CAPACITY REQUIRED, HIGH DEMAND or NORMAL and recommends an action.

Objectives of the case study:

- Analyse the requirements and identify entities, attributes, relationships and business rules.
- Design an ER model and convert it into a normalised relational schema with primary and foreign keys.
- Implement the schema with appropriate integrity constraints and load realistic, interrelated data.
- Implement booking, seat allocation, reservation modification and cancellation with refunds.
- Develop analytical SQL queries, advanced queries (joins, nested and correlated subqueries) and views.
- Build a decision-support module that identifies trains needing additional capacity.

Technology used: Oracle AI Database 26ai Free (release 23.26), SQL*Plus / Oracle SQL Developer for VS Code. The implementation is split into five scripts run in order by `run_all.sql`: `01_ddl.sql` (schema), `02_data.sql` (sample data), `02b_seat.sql` (seat generator), `03_dml.sql` (DML, integrity tests, procedures) and `04_queries.sql` (views, queries and the risk module).

2. Requirement Analysis

2.1 Functional Requirements

ID	Requirement
FR1	Maintain stations, routes and the ordered list of stops (with distance) on each route.
FR2	Maintain trains, the route each train runs on, its coach composition and every seat/berth in each coach.
FR3	Create a journey (schedule) for each date a train runs, with its status.
FR4	Register passengers with validated contact details and concession category.
FR5	Book a ticket: allocate a free seat of the requested class automatically, otherwise place the passenger on the waitlist.
FR6	Compute the fare from the travelled distance and class, and record the payment.
FR7	Cancel a ticket: compute the refund from the time left before departure, release the seat, and confirm the first waitlisted passenger.
FR8	Modify a reservation (e.g. change boarding point) with automatic fare recalculation.
FR9	Produce reports: train-wise reservations, station traffic, route/coach utilisation, cancellations, revenue, passenger frequency.
FR10	Classify trains as CAPACITY REQUIRED / HIGH DEMAND / NORMAL to support capacity planning.

2.2 Business Rules

ID	Rule	Enforced by
BR1	Station codes are unique upper-case codes.	PRIMARY KEY + CHECK
BR2	A route has different origin and destination; a station appears at most once on a route.	CHECK, UNIQUE(route_id, station_code)
BR3	A train runs on exactly one route; a route can have many trains.	FK train.route_id NOT NULL
BR4	Coach labels are unique within a train; every coach belongs to one class.	UNIQUE(train_no, coach_label), FK
BR5	A train runs at most once per date.	UNIQUE(train_no, journey_date)
BR6	Phone is a 10-digit Indian mobile number; phone and e-mail are unique.	CHECK (REGEXP_LIKE), UNIQUE
BR7	A seat can be sold only once per journey.	Function-based UNIQUE index
BR8	A passenger can hold only one active ticket per journey.	Function-based UNIQUE index
BR9	CONFIRMED/COMPLETED tickets must have a seat; WAITLIST/CANCELLED tickets must not.	CHECK ck_res_seat_status
BR10	The boarding station must come before the destination on the train's route.	Trigger
BR11	The allotted seat must belong to a coach of the same train and the booked class.	Trigger
BR12	A ticket cannot be booked after the journey date.	Trigger
BR13	Fare = max(distance, 50 km) × class fare per km + reservation fee.	Trigger
BR14	Every reservation has exactly one payment and at most one cancellation.	UNIQUE FK pnr
BR15	Refund: waitlist 100%, ≥ 48 h 90%, ≥ 24 h 75%, ≥ 4 h 50%, otherwise nil.	Procedure cancel_ticket
BR16	A passenger with booking history cannot be deleted.	FK without cascade

2.3 Assumptions

- One reservation (PNR) is one passenger on one seat; a group booking is several PNRs.
- A seat is held for the whole journey of the train; segment-wise seat reuse is out of scope.
- Seats per coach are scaled down (CC 12, SL 12, 3A 10, 2A 8, EC 8, 1A 4) so that occupancy patterns and waitlists are visible with a manageable data volume.
- A waitlisted ticket that is not confirmed by departure stays WAITLIST and is fully refunded; it is counted as unserved demand.

2.4 Entities and Attributes

Entity	Attributes (PK in bold)	Description
STATION	**station_code**, station_name, city, state, rail_zone, platforms	Railway station
ROUTE	**route_id**, route_name, origin_code, dest_code, distance_km	Corridor between two stations
ROUTE_STOP	**route_id, stop_seq**, station_code, km_from_origin	Ordered stops of a route (weak entity)
TRAIN	**train_no**, train_name, train_type, route_id, dep_time, run_days, is_active	Train service
COACH_CLASS	**class_code**, class_name, seats_per_coach, fare_per_km, reservation_fee	Travel class and fare rule
COACH	**coach_id**, train_no, coach_label, class_code	Coach attached to a train
SEAT	**seat_id**, coach_id, seat_no, berth_type	Seat or berth in a coach
TRAIN_SCHEDULE	**schedule_id**, train_no, journey_date, sched_status	One run of a train on a date
PASSENGER	**passenger_id**, full_name, gender, dob, phone, email, home_city, registered_on, concession_type	Registered traveller
RESERVATION	**pnr**, passenger_id, schedule_id, class_code, seat_id, from_station, to_station, booking_date, fare, status	Ticket; status is the ticket status
PAYMENT	**payment_id**, pnr, amount, pay_mode, paid_on, pay_status	Payment for a ticket
CANCELLATION	**cancel_id**, pnr, cancelled_on, refund_amount, reason	Cancellation and refund
RISK_THRESHOLD	**param_code**, param_value, description	Tunable thresholds of the risk rules
CAPACITY_RISK_LOG	**run_date, train_no**, occupancy_pct, cancellation_rate, demand_pct, risk_status	Daily risk classification (weak entity of TRAIN)

2.5 Relationships and Cardinalities

Relationship	Entities	Cardinality	Participation / note
Has stop (identifying)	ROUTE – ROUTE_STOP	1 : N	Total on ROUTE_STOP; stops deleted with the route
Located at	ROUTE_STOP – STATION	N : 1	Resolves the M : N between ROUTE and STATION
Starts at / Ends at	ROUTE – STATION	N : 1 (twice)	Two roles of STATION: origin and destination
Runs on	TRAIN – ROUTE	N : 1	Total on TRAIN
Has coach	TRAIN – COACH	1 : N	Total on COACH
Of class	COACH – COACH_CLASS	N : 1	Total on COACH
Contains	COACH – SEAT	1 : N	Total on SEAT
Runs as	TRAIN – TRAIN_SCHEDULE	1 : N	Total on TRAIN_SCHEDULE
Books	PASSENGER – RESERVATION	1 : N	Total on RESERVATION
For journey	TRAIN_SCHEDULE – RESERVATION	1 : N	Total on RESERVATION
Booked in	COACH_CLASS – RESERVATION	1 : N	Total on RESERVATION
Allotted	SEAT – RESERVATION	1 : N	Partial: waitlisted/cancelled

Relationship	Entities	Cardinality	Participation / note
			tickets have no seat; at most one ticket per seat per journey
Boards at / Alights at	STATION – RESERVATION	1 : N (twice)	Total on RESERVATION
Paid by	RESERVATION – PAYMENT	1 : 1	Every ticket has exactly one payment
Cancelled by	RESERVATION – CANCELLATION	1 : 0..1	Only cancelled tickets have a cancellation
Classified in (identifying)	TRAIN – CAPACITY_RISK_LOG	1 : N	One log entry per train per day

3. Database Design

3.1 ER Diagram

Figure 3.1 shows the ER diagram in Chen notation. Rectangles are entities (double rectangles are weak entities), diamonds are relationships (double diamonds are identifying relationships), ellipses are attributes (primary keys underlined, partial keys dashed), double lines show total participation and the labels 1 / N show cardinality. Foreign keys are not drawn as attributes in Chen notation; they appear as relationships.

The diagram is placed on the next page in landscape orientation for readability.

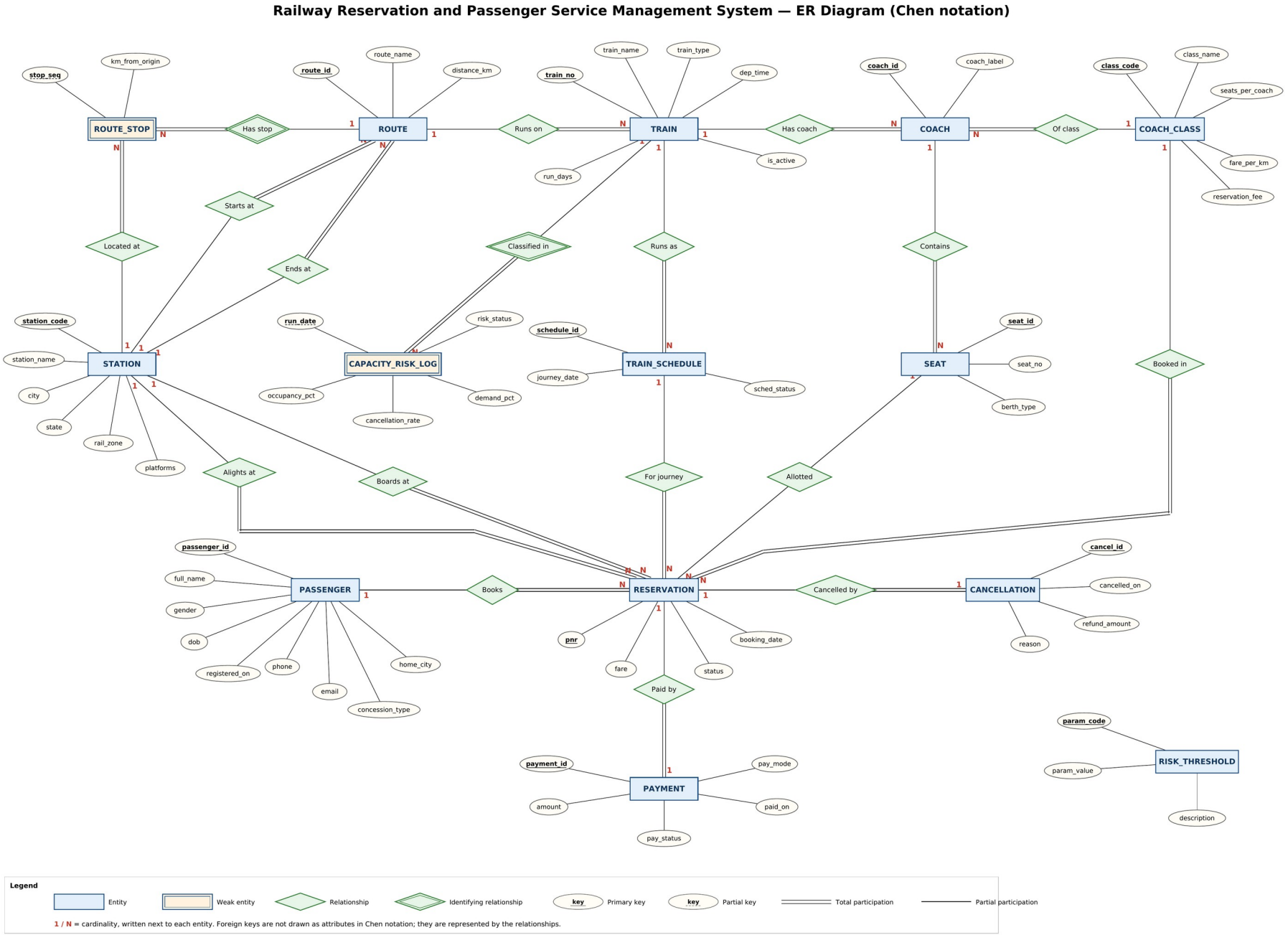


Figure 3.1: ER diagram of the Railway Reservation and Passenger Service Management System (Chen notation)

3.2 Relational Schema

The ER model is converted to relations as follows: every strong entity becomes a table with its key as the primary key; each 1 : N relationship becomes a foreign key on the N side; the 1 : 1 and 1 : 0..1 relationships (Paid by, Cancelled by) become a foreign key plus a UNIQUE constraint on the dependent side; the weak entities ROUTE_STOP and CAPACITY_RISK_LOG take the owner's key as part of a composite primary key; and the M : N relationship between ROUTE and STATION is resolved by ROUTE_STOP. Primary keys are shown in bold and foreign keys are marked with *.

- STATION(**station_code**, station_name, city, state, rail_zone, platforms)
- ROUTE(**route_id**, route_name, origin_code*, dest_code*, distance_km)
- ROUTE_STOP(**route_id***, **stop_seq**, station_code*, km_from_origin)
- TRAIN(**train_no**, train_name, train_type, route_id*, dep_time, run_days, is_active)
- COACH_CLASS(**class_code**, class_name, seats_per_coach, fare_per_km, reservation_fee)
- COACH(**coach_id**, train_no*, coach_label, class_code*)
- SEAT(**seat_id**, coach_id*, seat_no, berth_type)
- TRAIN_SCHEDULE(**schedule_id**, train_no*, journey_date, sched_status)
- PASSENGER(**passenger_id**, full_name, gender, dob, phone, email, home_city, registered_on, concession_type)
- RESERVATION(**pnr**, passenger_id*, schedule_id*, class_code*, seat_id*, from_station*, to_station*, booking_date, fare, status)
- PAYMENT(**payment_id**, pnr*, amount, pay_mode, paid_on, pay_status)
- CANCELLATION(**cancel_id**, pnr*, cancelled_on, refund_amount, reason)
- RISK_THRESHOLD(**param_code**, param_value, description)
- CAPACITY_RISK_LOG(**run_date**, **train_no***, occupancy_pct, cancellation_rate, demand_pct, risk_status)

3.3 Primary Keys and Foreign Keys

Table	Primary key	Foreign keys (→ referenced table)	Alternate keys (UNIQUE)
STATION	station_code	—	—
ROUTE	route_id	origin_code, dest_code → STATION	route_name
ROUTE_STOP	(route_id, stop_seq)	route_id → ROUTE (ON DELETE CASCADE); station_code → STATION	(route_id, station_code)
TRAIN	train_no	route_id → ROUTE	train_name
COACH_CLASS	class_code	—	—
COACH	coach_id	train_no → TRAIN (CASCADE); class_code → COACH_CLASS	(train_no, coach_label)
SEAT	seat_id	coach_id → COACH (CASCADE)	(coach_id, seat_no)
TRAIN_SCHEDULE	schedule_id	train_no → TRAIN	(train_no, journey_date)
PASSENGER	passenger_id	—	phone, email
RESERVATION	pnr	passenger_id → PASSENGER; schedule_id → TRAIN_SCHEDULE; class_code → COACH_CLASS; seat_id → SEAT; from_station, to_station → STATION	(schedule_id, seat_id)*, (passenger_id, schedule_id)* — function-based
PAYMENT	payment_id	pnr → RESERVATION (CASCADE)	pnr
CANCELLATION	cancel_id	pnr → RESERVATION (CASCADE)	pnr
RISK_THRESHOLD	param_code	—	—

Table	Primary key	Foreign keys (→ referenced table)	Alternate keys (UNIQUE)
CAPACITY_RISK_LOG	(run_date, train_no)	train_no → TRAIN	—

3.4 Normalisation

All twelve core relations are in **Boyce–Codd Normal Form (BCNF)**: every attribute is atomic (1NF), there are no partial dependencies on part of a composite key (2NF), and the determinant of every non-trivial functional dependency is a candidate key (3NF/BCNF). For example, train details depend only on train_no and are kept in TRAIN rather than repeated for every journey in TRAIN_SCHEDULE; station cities are joined from STATION rather than stored in ROUTE; and the distance of a trip is derived from ROUTE_STOP rather than stored in RESERVATION.

Two deliberate, controlled exceptions are documented: `reservation.fare` is stored (although it can be derived) so that a ticket's price does not change when class rates are revised later, and `reservation.class_code` is stored because waitlisted tickets have no seat from which the class could be derived. The trigger guarantees that both stay consistent.

4. Database Implementation

4.1 Table Creation using SQL

The schema is created by `01_ddl.sql`. A clean-up block first drops any existing project objects (using the data dictionary and dynamic SQL) so the script can be re-run. The two central tables are shown below; the other tables follow the same pattern.

```
-- 1.4 TRAIN : a train service that always runs on one route
CREATE TABLE train (
  train_no       NUMBER(5)     CONSTRAINT pk_train PRIMARY KEY
                 CONSTRAINT ck_train_no CHECK (train_no BETWEEN 10000 AND 99999),
  train_name     VARCHAR2(50)  NOT NULL,
  train_type     VARCHAR2(15)  NOT NULL
                 CONSTRAINT ck_train_type CHECK (train_type IN
('SUPERFAST','EXPRESS','SHATABDI','TEJAS','INTERCITY','MAIL')),
  route_id       NUMBER(4)     NOT NULL CONSTRAINT fk_train_route REFERENCES
route(route_id),
  dep_time       VARCHAR2(5)   NOT NULL
                 CONSTRAINT ck_train_dep CHECK (REGEXP_LIKE(dep_time, '^([01][0-9]|
2[0-3]):[0-5][0-9]$')),
  run_days       VARCHAR2(7)   DEFAULT 'YYYYYYY' NOT NULL
                 CONSTRAINT ck_train_days CHECK (REGEXP_LIKE(run_days, '^[YN]{7}$')),
  is_active      CHAR(1)       DEFAULT 'Y' NOT NULL CONSTRAINT ck_train_active CHECK
(is_active IN ('Y','N'))
);
```

```
-- 1.10 RESERVATION : one ticket (PNR) = one passenger on one journey; holds the
ticket status
CREATE TABLE reservation (
  pnr            NUMBER(10)    CONSTRAINT pk_reservation PRIMARY KEY,
  passenger_id   NUMBER(6)     NOT NULL CONSTRAINT fk_res_pass  REFERENCES
passenger(passenger_id),
  schedule_id    NUMBER(6)     NOT NULL CONSTRAINT fk_res_sched REFERENCES
train_schedule(schedule_id),
  class_code     VARCHAR2(3)   NOT NULL CONSTRAINT fk_res_class REFERENCES
coach_class(class_code),
  seat_id        NUMBER(7)     CONSTRAINT fk_res_seat REFERENCES seat(seat_id),
  from_station   VARCHAR2(5)   NOT NULL CONSTRAINT fk_res_from REFERENCES
station(station_code),
  to_station     VARCHAR2(5)   NOT NULL CONSTRAINT fk_res_to   REFERENCES
station(station_code),
  booking_date   DATE          DEFAULT TRUNC(SYSDATE) NOT NULL,
  fare           NUMBER(8,2)   CONSTRAINT ck_res_fare CHECK (fare > 0),
  status         VARCHAR2(10)  DEFAULT 'CONFIRMED' NOT NULL
                 CONSTRAINT ck_res_status CHECK (status IN
('CONFIRMED','WAITLIST','CANCELLED','COMPLETED')),
  CONSTRAINT ck_res_stations CHECK (from_station <> to_station),
  -- a confirmed / completed ticket must hold a seat; a waitlisted / cancelled one
must not
  CONSTRAINT ck_res_seat_status CHECK (
        (status IN ('CONFIRMED','COMPLETED') AND seat_id IS NOT NULL)
     OR (status IN ('WAITLIST','CANCELLED')  AND seat_id IS NULL))
);

-- No seat can be sold twice on the same journey (function-based unique index ignores
NULL seats)
CREATE UNIQUE INDEX ux_res_seat_journey
  ON reservation (CASE WHEN seat_id IS NOT NULL THEN schedule_id END, seat_id);
-- A passenger cannot hold two active tickets on the same journey
CREATE UNIQUE INDEX ux_res_pass_journey
  ON reservation (CASE WHEN status <> 'CANCELLED' THEN passenger_id END,
                  CASE WHEN status <> 'CANCELLED' THEN schedule_id  END);
CREATE INDEX ix_res_schedule_status ON reservation (schedule_id, status);

-- Supporting indexes on foreign keys to eliminate unindexed FK locking and speed up
joins
```

```
CREATE INDEX ix_res_passenger   ON reservation (passenger_id);
CREATE INDEX ix_res_from_stn    ON reservation (from_station);
CREATE INDEX ix_res_to_stn      ON reservation (to_station);
CREATE INDEX ix_res_seat        ON reservation (seat_id);
CREATE INDEX ix_rs_station      ON route_stop (station_code);
CREATE INDEX ix_train_route     ON train (route_id);
CREATE INDEX ix_coach_class     ON coach (class_code);
```

Sequences generate new PNRs, payment ids and cancellation ids:

```
CREATE SEQUENCE seq_pnr     START WITH 4200001001 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_payment START WITH 901001     INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_cancel  START WITH 701001     INCREMENT BY 1 NOCACHE;
```

Table alteration (ALTER TABLE)

```
-- 3.1 add a concession category to passengers (with domain constraint)
ALTER TABLE passenger ADD (
  concession_type VARCHAR2(10) DEFAULT 'NONE' NOT NULL
  CONSTRAINT ck_pass_concession CHECK (concession_type IN
('NONE','SENIOR','STUDENT','DIVYANG'))
);
-- 3.2 widen a column
ALTER TABLE station MODIFY (station_name VARCHAR2(80));
-- 3.3 rename a column to a clearer name
ALTER TABLE passenger RENAME COLUMN city TO home_city;
-- 3.4 add a table-level constraint after creation
ALTER TABLE train ADD CONSTRAINT uq_train_name UNIQUE (train_name);
```

These statements add a mandatory column with a default (existing rows receive 'NONE'), widen a column, rename a column, and add a UNIQUE constraint to a populated table.

4.2 Integrity Constraints

Constraint type	Examples in the project	Purpose
PRIMARY KEY	pk_station, pk_route_stop (composite), pk_reservation	Entity integrity: every row uniquely identified
FOREIGN KEY	fk_res_pass, fk_res_sched, fk_res_seat, fk_rs_station	Referential integrity between related tables
ON DELETE CASCADE	fk_rs_route, fk_coach_train, fk_seat_coach, fk_pay_res, fk_cancel_res	Dependent rows removed with their parent
UNIQUE	uq_route_name, uq_sched (train_no, journey_date), uq_pass_phone, uq_pay_pnr	Alternate keys; one-to-one relationships
CHECK	ck_station_code (uppercase), ck_train_dep (HH:MM regex), ck_pass_phone (regex), ck_sched_date, ck_res_seat_status	Domain and row-level business rules
NOT NULL / DEFAULT	status DEFAULT 'CONFIRMED', booking_date DEFAULT TRUNC(SYSDATE), platforms DEFAULT 1	Mandatory values and sensible defaults
Function-based UNIQUE index	ux_res_seat_journey, ux_res_pass_journey	No seat sold twice per journey; one active ticket per passenger per journey
Trigger	trg_reservation_validate (BEFORE INSERT OR UPDATE, FOR EACH ROW)	Multi-table rules R1–R4: direction, seat ownership, booking date, fare calculation

The function-based index that prevents double booking ignores tickets without a seat, so any number of waitlisted tickets can exist:

```
-- No seat can be sold twice on the same journey (function-based unique index ignores
```

```
NULL seats)
CREATE UNIQUE INDEX ux_res_seat_journey
  ON reservation (CASE WHEN seat_id IS NOT NULL THEN schedule_id END, seat_id);
-- A passenger cannot hold two active tickets on the same journey
CREATE UNIQUE INDEX ux_res_pass_journey
  ON reservation (CASE WHEN status <> 'CANCELLED' THEN passenger_id END,
                  CASE WHEN status <> 'CANCELLED' THEN schedule_id  END);
```

The validation trigger calculates the fare and enforces the rules that involve several tables:

```
-- 1.13 TRIGGER : business rules that span several tables (cannot be CHECK
constraints)
--   R1 boarding station must come before the destination on the train's route
--   R2 the seat must belong to a coach of THIS train and of the booked class
--   R3 a ticket cannot be booked after the journey date
--   R4 fare = max(distance, 50 km) * fare_per_km + reservation fee  (auto-calculated)
--
-------------------------------------------------------------------------------
CREATE OR REPLACE TRIGGER trg_reservation_validate
BEFORE INSERT OR UPDATE OF schedule_id, class_code, seat_id, from_station, to_station,
fare
ON reservation
FOR EACH ROW
DECLARE
  v_train     train.train_no%TYPE;
  v_route     train.route_id%TYPE;
  v_jdate     train_schedule.journey_date%TYPE;
  v_from_seq  route_stop.stop_seq%TYPE;
  v_to_seq    route_stop.stop_seq%TYPE;
  v_from_km   route_stop.km_from_origin%TYPE;
  v_to_km     route_stop.km_from_origin%TYPE;
  v_seat_trn  train.train_no%TYPE;
  v_seat_cls  coach_class.class_code%TYPE;
  v_rate      coach_class.fare_per_km%TYPE;
  v_fee       coach_class.reservation_fee%TYPE;
BEGIN
  SELECT t.train_no, t.route_id, s.journey_date
    INTO v_train, v_route, v_jdate
    FROM train_schedule s JOIN train t ON t.train_no = s.train_no
   WHERE s.schedule_id = :NEW.schedule_id;

  -- R1
  BEGIN
    SELECT stop_seq, km_from_origin INTO v_from_seq, v_from_km
      FROM route_stop WHERE route_id = v_route AND station_code = :NEW.from_station;
    SELECT stop_seq, km_from_origin INTO v_to_seq, v_to_km
      FROM route_stop WHERE route_id = v_route AND station_code = :NEW.to_station;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20001, 'Station is not on the route of train ' ||
v_train);
  END;
  IF v_from_seq >= v_to_seq THEN
    RAISE_APPLICATION_ERROR(-20002, 'Boarding station must come before destination on
the route');
  END IF;

  -- R2
  IF :NEW.seat_id IS NOT NULL THEN
    SELECT c.train_no, c.class_code INTO v_seat_trn, v_seat_cls
      FROM seat st JOIN coach c ON c.coach_id = st.coach_id
     WHERE st.seat_id = :NEW.seat_id;
    IF v_seat_trn <> v_train OR v_seat_cls <> :NEW.class_code THEN
      RAISE_APPLICATION_ERROR(-20003, 'Seat ' || :NEW.seat_id || ' does not belong to
train '
                              || v_train || ' / class ' || :NEW.class_code);
    END IF;
  END IF;

  -- R3
  IF INSERTING AND :NEW.booking_date > v_jdate THEN
```

```
    RAISE_APPLICATION_ERROR(-20004, 'Booking date cannot be after the journey date');
  END IF;

  -- R4
  IF :NEW.fare IS NULL THEN
    SELECT fare_per_km, reservation_fee INTO v_rate, v_fee
      FROM coach_class WHERE class_code = :NEW.class_code;
    :NEW.fare := ROUND(GREATEST(v_to_km - v_from_km, 50) * v_rate + v_fee, 0);
  END IF;
END;
/
```

4.3 Data Insertion

`02_data.sql` loads realistic, interrelated data in parent-to-child order. Seats are generated in one INSERT … SELECT from the coach and class data, and the ticket fares are calculated by the trigger. Payments and cancellations are derived from the reservations with INSERT … SELECT.

```
INSERT INTO station VALUES ('MAS', 'Chennai Central', 'Chennai', 'Tamil Nadu', 'SR',
17);
INSERT INTO route VALUES (1, 'Chennai - Bengaluru', 'MAS', 'SBC', 359);
INSERT INTO train (train_no, train_name, train_type, route_id, dep_time) VALUES
(12639, 'Brindavan Express', 'SUPERFAST', 1, '07:50');
INSERT INTO passenger VALUES (1, 'Balaji Mani', 'M', DATE '1993-07-10', '7624642010',
'balaji.mani1@mail.com', 'Kochi', DATE '2025-01-28');
-- fare is not supplied: the trigger computes it
INSERT INTO reservation (pnr, passenger_id, schedule_id, class_code, seat_id,
from_station, to_station, booking_date, status)
VALUES (4200000001, 11, 101, 'CC', 205, 'MAS', 'SBC', DATE '2026-08-11', 'COMPLETED');
```

```
INSERT INTO seat (seat_id, coach_id, seat_no, berth_type)
SELECT c.coach_id * 100 + n.lvl, c.coach_id, n.lvl,
       CASE
         WHEN c.class_code IN ('SL','3A') THEN
              DECODE(MOD(n.lvl, 8), 1,'LOWER', 2,'MIDDLE', 3,'UPPER', 4,'LOWER',
5,'MIDDLE', 6,'UPPER', 7,'SIDE LOWER', 'SIDE UPPER')
         WHEN c.class_code = '2A' THEN
              DECODE(MOD(n.lvl, 6), 1,'LOWER', 2,'UPPER', 3,'LOWER', 4,'UPPER',
5,'SIDE LOWER', 'SIDE UPPER')
         WHEN c.class_code = '1A' THEN DECODE(MOD(n.lvl, 2), 1, 'LOWER', 'UPPER')
         WHEN c.class_code = 'EC' THEN DECODE(MOD(n.lvl, 2), 1, 'WINDOW', 'AISLE')
         ELSE DECODE(MOD(n.lvl, 3), 1, 'WINDOW', 2, 'MIDDLE SEAT', 'AISLE')
       END
FROM   coach c
       JOIN coach_class cc ON cc.class_code = c.class_code
       JOIN (SELECT LEVEL AS lvl FROM dual CONNECT BY LEVEL <= 100) n ON n.lvl <=
cc.seats_per_coach;
```

Row counts after the complete run (including the booking demo):

TBL	ROW_COUNT
STATION	14
ROUTE	4
ROUTE_STOP	21
TRAIN	9
COACH_CLASS	6
COACH	30
SEAT	320
TRAIN_SCHEDULE	28
PASSENGER	150
RESERVATION	777

TBL	ROW_COUNT
PAYMENT	777
CANCELLATION	36

4.4 UPDATE and DELETE Operations

Update: concession category from age

```
-- 4.1 derive concession category from age
UPDATE passenger SET concession_type = 'SENIOR'  WHERE MONTHS_BETWEEN(SYSDATE, dob) /
12 >= 60;
UPDATE passenger SET concession_type = 'STUDENT' WHERE MONTHS_BETWEEN(SYSDATE, dob) /
12 < 23;
```

Result: 9 passengers were marked SENIOR and 17 STUDENT (ages computed on the run date).

Update: timetable revision

```
-- 4.2 timetable revision
UPDATE train SET dep_time = '07:40' WHERE train_no = 12639;
```

Reservation modification: change of boarding point

The passenger changes the boarding point from Chennai (MAS) to Katpadi (KPD). Setting the fare to NULL makes the trigger recalculate it for the shorter distance, and the payment is updated to match.

```
UPDATE reservation SET from_station = 'KPD', fare = NULL WHERE pnr = :v_pnr;
UPDATE payment SET amount = (SELECT fare FROM reservation WHERE pnr = :v_pnr) WHERE
pnr = :v_pnr;
```

	PNR	FROM	TO	FARE	PAID
Before	4200000071	MAS	SBC	561	561
After	4200000071	KPD	SBC	374	374

Delete

```
-- 4.4 DELETE : a registered passenger who never booked can be removed
INSERT INTO passenger (passenger_id, full_name, gender, dob, phone, email, home_city)
VALUES (999, 'Test Account', 'M', DATE '1999-05-05', '9000000001',
'test.account@mail.com', 'Chennai');
DELETE FROM passenger WHERE passenger_id = 999;
```

A passenger with no bookings can be deleted; a passenger with bookings cannot (see 4.5).

4.5 Referential Integrity and Constraint Enforcement

Part 5 of `03_dml.sql` deliberately attempts seven invalid operations. Each is rejected by the database and the error is printed:

Test	Invalid operation	Result (error raised)
5.1	Passenger with phone '12345'	ORA-02290: check constraint CK_PASS_PHONE violated

Test	Invalid operation	Result (error raised)
5.2	Sell a seat already sold on journey 101	ORA-00001: unique constraint UX_RES_SEAT_JOURNEY violated
5.3	Seat of train 12677 on a journey of 12639	ORA-20003: Seat 2801 does not belong to train 12639 / class CC
5.4	Boarding SBC, destination MAS (wrong direction)	ORA-20002: Boarding station must come before destination on the route
5.5	CONFIRMED ticket without a seat	ORA-02290: check constraint CK_RES_SEAT_STATUS violated
5.6	Delete passenger 1 who has tickets	ORA-02292: integrity constraint FK_RES_PASS violated - child record found
5.7	Route stop for unknown station 'XYZ'	ORA-02291: integrity constraint FK_RS_STATION violated - parent key not found

4.6 Booking and Ticket Cancellation (Stored Procedures)

book_ticket validates the passenger, journey and class, locks the journey row (SELECT … FOR UPDATE) so that concurrent bookings of the same journey are serialised, allocates the lowest-numbered free seat of the requested class or places the passenger on the waitlist, inserts the reservation (the trigger computes the fare) and records the payment. If a seat is taken concurrently, the unique index raises DUP_VAL_ON_INDEX and the procedure retries with the next seat.

cancel_ticket locks the ticket, computes the exact departure time and the refund (waitlist 100%, ≥ 48 h 90%, ≥ 24 h 75%, ≥ 4 h 50%, otherwise nil), marks the ticket CANCELLED and releases its seat, records the cancellation, updates the payment status and promotes the earliest waitlisted passenger of the same journey and class to the released seat.

```
-- 6.2 CANCEL_TICKET : refund by time-to-departure, release seat, promote first
waitlisted ticket
--     Refund : waitlist 100% | >=48 h 90% | >=24 h 75% | >=4 h 50% | <4 h nil
--     Hardened with: PNR existence check, serialized waitlist row-level lock (FOR
UPDATE)
CREATE OR REPLACE PROCEDURE cancel_ticket (p_pnr IN NUMBER, p_reason IN VARCHAR2) AS
  v_status   reservation.status%TYPE;
  v_fare     reservation.fare%TYPE;
  v_seat     reservation.seat_id%TYPE;
  v_sched    reservation.schedule_id%TYPE;
  v_class    reservation.class_code%TYPE;
  v_depart   DATE;
  v_hours    NUMBER;
  v_refund   NUMBER(8,2);
  v_promoted reservation.pnr%TYPE;
BEGIN
  BEGIN
    SELECT r.status, r.fare, r.seat_id, r.schedule_id, r.class_code,
           s.journey_date + TO_NUMBER(SUBSTR(t.dep_time, 1, 2)) / 24 +
TO_NUMBER(SUBSTR(t.dep_time, 4, 2)) / 1440
    INTO   v_status, v_fare, v_seat, v_sched, v_class, v_depart
    FROM   reservation r
           JOIN train_schedule s ON s.schedule_id = r.schedule_id
           JOIN train t          ON t.train_no    = s.train_no
    WHERE  r.pnr = p_pnr
    FOR UPDATE OF r.status;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20016, 'PNR ' || p_pnr || ' does not exist');
  END;

  IF v_status NOT IN ('CONFIRMED','WAITLIST') THEN
    RAISE_APPLICATION_ERROR(-20012, 'PNR ' || p_pnr || ' is ' || v_status || ' and
cannot be cancelled');
```

```
  END IF;
  v_hours := (v_depart - SYSDATE) * 24;
  IF v_hours < 0 THEN
    RAISE_APPLICATION_ERROR(-20013, 'Train has already departed');
  END IF;

  v_refund := ROUND(v_fare * CASE WHEN v_status = 'WAITLIST' THEN 1
                                  WHEN v_hours >= 48 THEN 0.90
                                  WHEN v_hours >= 24 THEN 0.75
                                  WHEN v_hours >= 4  THEN 0.50
                                  ELSE 0 END, 2);

  UPDATE reservation SET status = 'CANCELLED', seat_id = NULL WHERE pnr = p_pnr;
  INSERT INTO cancellation VALUES (seq_cancel.NEXTVAL, p_pnr, TRUNC(SYSDATE),
v_refund, p_reason);
  UPDATE payment SET pay_status = CASE WHEN v_refund >= amount THEN 'REFUNDED'
                                       WHEN v_refund > 0 THEN 'PARTIAL_REFUND'
                                       ELSE 'SUCCESS' END
  WHERE  pnr = p_pnr;
  DBMS_OUTPUT.PUT_LINE('PNR ' || p_pnr || ' cancelled, refund Rs.' || v_refund);

  -- Promote earliest waitlisted passenger in FIFO order (loop guards against
concurrent promotion)
  IF v_seat IS NOT NULL THEN
    LOOP
      BEGIN
        SELECT pnr INTO v_promoted
        FROM   reservation
        WHERE  schedule_id = v_sched
        AND    class_code  = v_class
        AND    status      = 'WAITLIST'
        ORDER  BY booking_date, pnr
        FETCH FIRST 1 ROW ONLY;

        UPDATE reservation
        SET    status = 'CONFIRMED', seat_id = v_seat
        WHERE  pnr = v_promoted
        AND    status = 'WAITLIST';

        IF SQL%ROWCOUNT > 0 THEN
          DBMS_OUTPUT.PUT_LINE('Waitlisted PNR ' || v_promoted || ' confirmed on
released seat ' || v_seat);
          EXIT;
        END IF;
      EXCEPTION
        WHEN NO_DATA_FOUND THEN
          EXIT;
      END;
    END LOOP;
  END IF;
END;
/
```

Demonstration: nine passengers book the eight-seat Executive Chair Car of a journey seven days ahead; the ninth is waitlisted. Passenger 11 then cancels, receives a 90% refund, and the waitlisted passenger is confirmed automatically on the released seat.

```
Passenger 11 -> PNR 4200001001 CONFIRMED
...
Passenger 18 -> PNR 4200001008 CONFIRMED
Passenger 19 -> PNR 4200001009 WAITLIST
PNR 4200001001 cancelled, refund Rs.1588.5
Waitlisted PNR 4200001009 confirmed on released seat 3001
```

PNR	FULL_NAME	SEAT	STATUS	FARE	PAY_STATUS	REFUND_AMOUNT
4200001001	Padma Kumar	-	CANCELLED	1765	PARTIAL_REFUND	1588.5

PNR	FULL_NAME	SEAT	STATUS	FARE	PAY_STATUS	REFUND_AMOUNT
4200001002	Deepa Anand	E1-2	CONFIRMED	1765	SUCCESS	NULL
4200001003	Charulatha Sundar	E1-3	CONFIRMED	1765	SUCCESS	NULL
4200001004	Asha Mani	E1-4	CONFIRMED	1765	SUCCESS	NULL
4200001005	Ajay Reddy	E1-5	CONFIRMED	1765	SUCCESS	NULL
4200001006	Krishna Velu	E1-6	CONFIRMED	1765	SUCCESS	NULL
4200001007	Revathi Selvam	E1-7	CONFIRMED	1765	SUCCESS	NULL
4200001008	Asha Hegde	E1-8	CONFIRMED	1765	SUCCESS	NULL
4200001009	Sangeetha Mani	E1-1	CONFIRMED	1765	SUCCESS	NULL

5. SQL Queries and Outputs

All outputs below were produced by running the project's queries against the database after a complete run of `run_all.sql`.

5.1 Train-wise Reservation Statistics

```
-- Q1 TRAIN-WISE RESERVATION STATISTICS  (conditional aggregation / pivot)
SELECT t.train_no, t.train_name,
       COUNT(*)                                                  AS bookings,
       COUNT(CASE WHEN r.status = 'COMPLETED' THEN 1 END)        AS completed,
       COUNT(CASE WHEN r.status = 'CONFIRMED' THEN 1 END)        AS confirmed,
       COUNT(CASE WHEN r.status = 'WAITLIST' THEN 1 END)         AS waitlist,
       COUNT(CASE WHEN r.status = 'CANCELLED' THEN 1 END)        AS cancelled,
       ROUND(AVG(r.fare), 2)                                     AS avg_fare,
       MIN(r.booking_date)                                       AS first_booking,
       MAX(r.booking_date)                                       AS last_booking
FROM   reservation r
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
GROUP  BY t.train_no, t.train_name
ORDER  BY bookings DESC;
```

TRAIN_NO	TRAIN_NAME	BOOKINGS	COMPLETED	CONFIRMED	WAITLIST	CANCELLED	AVG_FARE	FIRST_BOOKING	LAST_BOOKING
12637	Pandian Express	131	83	39	8	1	548.83	10-AUG-2026	18-OCT-2026
12027	Chennai Shatabdi	112	63	30	5	14	617.38	08-AUG-2026	17-OCT-2026
12639	Brindavan Express	102	63	30	7	2	562.35	11-AUG-2026	18-OCT-2026
12635	Vaigai Express	93	61	28	0	4	653.16	12-AUG-2026	17-OCT-2026
12671	Nilgiri Express	85	56	26	0	3	648.32	14-AUG-2026	18-OCT-2026
12675	Kovai Express	78	51	24	0	3	796.67	12-AUG-2026	19-OCT-2026
12607	Lalbagh Express	71	47	22	0	2	511.14	09-AUG-2026	19-OCT-2026
12677	Ernakulam Intercity	54	30	21	0	3	1110.26	15-AUG-2026	28-OCT-2026
22671	Madurai Tejas	51	33	14	0	4	760.29	11-AUG-2026	15-OCT-2026

Observation: Bookings per train with a count for each ticket status (conditional aggregation), average fare and the booking window. Pandian Express (12637) has the most bookings; Chennai Shatabdi has the most cancellations.

5.2 Station-wise Passenger Volume

```
-- Q2 STATION-WISE PASSENGER VOLUME  (boarding + alighting, stations with no traffic
included)
SELECT st.station_code, st.station_name, st.city,
       NVL(b.boarding, 0)                       AS boarded,
       NVL(a.alighting, 0)                      AS alighted,
       NVL(b.boarding, 0) + NVL(a.alighting, 0)  AS total_footfall
FROM   station st
       LEFT JOIN (SELECT from_station AS code, COUNT(*) AS boarding FROM reservation
                  WHERE status IN ('CONFIRMED','COMPLETED') GROUP BY from_station) b
ON b.code = st.station_code
       LEFT JOIN (SELECT to_station AS code, COUNT(*) AS alighting FROM reservation
                  WHERE status IN ('CONFIRMED','COMPLETED') GROUP BY to_station) a
ON a.code = st.station_code
ORDER  BY total_footfall DESC;
```

STATION_CODE	STATION_NAME	CITY	BOARDED	ALIGHTED	TOTAL_FOOTFALL
MAS	Chennai Central	Chennai	557	0	557
SBC	KSR Bengaluru City	Bengaluru	43	228	271
MDU	Madurai Junction	Madurai	0	226	226
CBE	Coimbatore Junction	Coimbatore	2	143	145
ERS	Ernakulam Junction	Kochi	0	45	45
KPD	Katpadi Junction	Vellore	32	10	42
JTJ	Jolarpettai Junction	Jolarpettai	22	18	40
DG	Dindigul Junction	Dindigul	18	20	38
ED	Erode Junction	Erode	11	11	22
TPJ	Tiruchchirappalli Junction	Tiruchirappalli	14	6	20
VM	Villupuram Junction	Villupuram	11	6	17
SA	Salem Junction	Salem	10	6	16
PGT	Palakkad Junction	Palakkad	1	2	3
TVC	Thiruvananthapuram Central	Thiruvananthapuram	0	0	0

Observation: Boardings and alightings per station using two pre-aggregated subqueries and LEFT JOINs, so stations without traffic (TVC) still appear. Chennai Central is the busiest station.

5.3 Route Utilization

```
-- Q3 ROUTE UTILIZATION
SELECT ro.route_name, ro.distance_km,
       COUNT(*)                                              AS trains,
       SUM(u.journeys)                                       AS journeys,
       SUM(u.capacity_offered)                               AS seats_offered,
       SUM(u.seats_occupied)                                 AS seats_sold,
       ROUND(100 * SUM(u.seats_occupied) / SUM(u.capacity_offered), 2) AS
route_occupancy_pct,
       SUM(u.revenue)                                        AS revenue
FROM   v_train_utilization u JOIN route ro ON ro.route_id = u.route_id
GROUP  BY ro.route_name, ro.distance_km
ORDER  BY route_occupancy_pct DESC;
```

ROUTE_NAME	DISTANCE_KM	TRAINS	JOURNEYS	SEATS_OFFERED	SEATS_SOLD	ROUTE_OCCUPANCY_PCT	REVENUE
Chennai - Bengaluru	359	3	9	300	255	85	148033.7
Chennai - Madurai	493	3	9	330	258	78.18	163561.9
Chennai - Coimbatore	494	2	6	234	157	67.09	114262.6
Bengaluru - Ernakulam	588	1	4	128	51	39.84	55973.3

Observation: Seats offered and sold per route from the train-utilization view. Chennai–Bengaluru is the most utilised corridor (85%).

5.4 Coach Occupancy

```
-- Q4 COACH OCCUPANCY  (per coach across all journeys of its train)
SELECT t.train_no, t.train_name, c.coach_label, c.class_code,
       cc.seats_per_coach * j.journeys                        AS seat_journeys,
       COUNT(r.pnr)                                           AS occupied,
```

```
       ROUND(100 * COUNT(r.pnr) / (cc.seats_per_coach * j.journeys), 1) AS
occupancy_pct
FROM   coach c
       JOIN coach_class cc ON cc.class_code = c.class_code
       JOIN train t        ON t.train_no = c.train_no
       JOIN (SELECT train_no, COUNT(*) AS journeys FROM train_schedule GROUP BY
train_no) j
                           ON j.train_no = c.train_no
       LEFT JOIN seat st   ON st.coach_id = c.coach_id
       LEFT JOIN reservation r ON r.seat_id = st.seat_id AND r.status IN
('CONFIRMED','COMPLETED')
GROUP  BY t.train_no, t.train_name, c.coach_label, c.class_code, cc.seats_per_coach,
j.journeys
ORDER  BY t.train_no, c.coach_label;
```

TRAIN_NO	TRAIN_NAME	COACH_LABEL	CLASS_CODE	SEAT_JOURNEYS	OCCUPIED	OCCUPANCY_PCT
12027	Chennai Shatabdi	C1	CC	36	35	97.2
12027	Chennai Shatabdi	C2	CC	36	36	100
12027	Chennai Shatabdi	E1	EC	24	22	91.7
12607	Lalbagh Express	C1	CC	36	22	61.1
12607	Lalbagh Express	C2	CC	36	25	69.4
12607	Lalbagh Express	C3	CC	36	22	61.1
12635	Vaigai Express	C1	CC	36	29	80.6
12635	Vaigai Express	C2	CC	36	30	83.3
12635	Vaigai Express	C3	CC	36	30	83.3
12637	Pandian Express	A1	2A	24	23	95.8
12637	Pandian Express	B1	3A	30	29	96.7
12637	Pandian Express	S1	SL	36	35	97.2

(30 rows returned; first 12 shown.)

Observation: Occupancy of every coach over all journeys of its train. The ticket filter is placed in the ON clause so that unsold seats are kept.

5.5 Cancellation Statistics

```
-- Q5a CANCELLATION STATISTICS BY TRAIN
SELECT t.train_no, t.train_name,
       COUNT(r.pnr)                                              AS bookings,
       COUNT(cn.cancel_id)                                       AS cancellations,
       ROUND(100 * COUNT(cn.cancel_id) / COUNT(r.pnr), 2)        AS cancel_rate_pct,
       NVL(SUM(cn.refund_amount), 0)                             AS refunded,
       NVL(SUM(CASE WHEN cn.cancel_id IS NOT NULL THEN r.fare - cn.refund_amount END),
0) AS charges_retained,
       ROUND(AVG(s.journey_date - cn.cancelled_on), 1)           AS
avg_days_before_journey
FROM   reservation r
       JOIN train_schedule s     ON s.schedule_id = r.schedule_id
       JOIN train t              ON t.train_no = s.train_no
       LEFT JOIN cancellation cn ON cn.pnr = r.pnr
GROUP  BY t.train_no, t.train_name
ORDER  BY cancel_rate_pct DESC;

-- Q5b CANCELLATION REASONS  (share of total using RATIO_TO_REPORT)
SELECT reason, COUNT(*) AS cancellations,
       ROUND(100 * RATIO_TO_REPORT(COUNT(*)) OVER (), 1) AS share_pct,
       SUM(refund_amount) AS refunded
FROM   cancellation
GROUP  BY reason
ORDER  BY cancellations DESC;
```

TRAIN_NO	TRAIN_NAME	BOOKINGS	CANCELLATIONS	CANCEL_RATE_PCT	REFUNDED	CHARGES_RETAINED	AVG_DAYS_BEFORE_JOURNEY
12027	Chennai Shatabdi	112	14	12.5	6958.8	773.2	13.1
22671	Madurai Tejas	51	4	7.84	2609.1	289.9	13.3
12677	Ernakulam Intercity	54	3	5.56	3980.7	442.3	12
12635	Vaigai Express	93	4	4.3	2718	302	16.3
12675	Kovai Express	78	3	3.85	1632.6	181.4	19.3
12671	Nilgiri Express	85	3	3.53	1351.8	150.2	9
12607	Lalbagh Express	71	2	2.82	1009.8	112.2	16
12639	Brindavan Express	102	2	1.96	731.7	81.3	12
12637	Pandian Express	131	1	0.76	306	34	13

REASON	CANCELLATIONS	SHARE_PCT	REFUNDED
Change of plans	10	27.8	6794.1
Booked alternate travel	9	25	5880.6
Official work	9	25	4386.6
Medical emergency	8	22.2	4237.2

Observation: Cancellation rate, refunds and charges retained per train, and the share of each cancellation reason (RATIO_TO_REPORT). Chennai Shatabdi has the highest cancellation rate (12.5%).

5.6 Peak Travel Routes

```
-- Q6a PEAK TRAVEL ROUTES  (ranked by passengers carried)
SELECT ro.route_name,
       COUNT(*)                                      AS passengers,
       DENSE_RANK() OVER (ORDER BY COUNT(*) DESC)   AS demand_rank
FROM   reservation r
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
       JOIN route ro         ON ro.route_id = t.route_id
WHERE  r.status IN ('CONFIRMED','COMPLETED','WAITLIST')
GROUP  BY ro.route_name;

-- Q6b TOP 5 ORIGIN-DESTINATION PAIRS (peak travel segments)
SELECT fs.city || ' -> ' || ts.city AS segment, COUNT(*) AS passengers
FROM   reservation r
       JOIN station fs ON fs.station_code = r.from_station
       JOIN station ts ON ts.station_code = r.to_station
WHERE  r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY fs.city, ts.city
ORDER  BY passengers DESC
FETCH FIRST 5 ROWS ONLY;

-- Q6c PEAK TRAVEL DAYS OF THE WEEK
SELECT TO_CHAR(s.journey_date, 'DY') AS travel_day,
       COUNT(*)                      AS passengers,
       ROUND(100 * RATIO_TO_REPORT(COUNT(*)) OVER (), 1) AS share_pct
FROM   reservation r JOIN train_schedule s ON s.schedule_id = r.schedule_id
WHERE  r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY TO_CHAR(s.journey_date, 'DY')
ORDER  BY passengers DESC;
```

ROUTE_NAME	PASSENGERS	DEMAND_RANK
Chennai - Bengaluru	267	1

ROUTE_NAME	PASSENGERS	DEMAND_RANK
Chennai - Madurai	266	2
Chennai - Coimbatore	157	3
Bengaluru - Ernakulam	51	4

SEGMENT	PASSENGERS
Chennai -> Madurai	201
Chennai -> Bengaluru	195
Chennai -> Coimbatore	125
Bengaluru -> Kochi	41
Dindigul -> Madurai	18

TRAVEL_DAY	PASSENGERS	SHARE_PCT
SUN	158	21.9
THU	129	17.9
WED	113	15.7
SAT	92	12.8
FRI	89	12.3
MON	77	10.7
TUE	63	8.7

Observation: Routes ranked by passenger demand (DENSE_RANK), the five busiest city pairs and the busiest days of the week.

5.7 Highest and Lowest Utilized Trains

```
-- Q7 HIGHEST AND LOWEST UTILIZED TRAINS
SELECT CASE WHEN rk_high = 1 THEN 'HIGHEST' ELSE 'LOWEST' END AS position,
       train_no, train_name, route_label, occupancy_pct
FROM  (SELECT u.*, RANK() OVER (ORDER BY occupancy_pct DESC) AS rk_high,
                   RANK() OVER (ORDER BY occupancy_pct ASC)  AS rk_low
       FROM   v_train_utilization u)
WHERE  rk_high = 1 OR rk_low = 1;
```

POSITION	TRAIN_NO	TRAIN_NAME	ROUTE_LABEL	OCCUPANCY_PCT
LOWEST	12677	Ernakulam Intercity	Bengaluru-Kochi	39.84
HIGHEST	12027	Chennai Shatabdi	Chennai-Bengaluru	96.88
HIGHEST	12639	Brindavan Express	Chennai-Bengaluru	96.88

Observation: Two RANK() window functions in an inline view; ties are kept. Brindavan Express and Chennai Shatabdi share the highest occupancy (96.88%); Ernakulam Intercity is the least used (39.84%).

5.8 Passenger Travel Frequency

```
-- Q8 PASSENGER TRAVEL FREQUENCY  (trips, distance, spend, banding with CASE)
SELECT p.passenger_id, p.full_name, p.home_city,
       COUNT(*)                    AS trips,
       SUM(v.distance_km)          AS km_travelled,
       SUM(v.fare)                 AS total_spent,
       CASE WHEN COUNT(*) >= 6 THEN 'FREQUENT'
            WHEN COUNT(*) >= 3 THEN 'REGULAR'
            ELSE 'OCCASIONAL' END   AS traveller_type
FROM   v_ticket_details v JOIN passenger p ON p.passenger_id = v.passenger_id
WHERE  v.status IN ('CONFIRMED','COMPLETED')
GROUP  BY p.passenger_id, p.full_name, p.home_city
ORDER  BY trips DESC, total_spent DESC;
```

PASSENGER_ID	FULL_NAME	HOME_CITY	TRIPS	KM_TRAVELLED	TOTAL_SPENT	TRAVELLER_TYPE
3	Nithya Balaji	Chennai	14	5187	8957	FREQUENT
15	Ajay Reddy	Chennai	13	5332	11039	FREQUENT
6	Janani Shankar	Bengaluru	12	4781	7515	FREQUENT
13	Charulatha Sundar	Bengaluru	11	4154	8358	FREQUENT
19	Sangeetha Mani	Dindigul	11	4207	7897	FREQUENT
8	Kishore Kumar	Kochi	11	3634	5662	FREQUENT
64	Krishna Rajan	Chennai	10	4246	6788	FREQUENT
9	Fathima Elango	Chennai	10	4049	6720	FREQUENT
5	Priya Pandian	Tiruchirappalli	10	3402	5116	FREQUENT
2	Ajay Thomas	Vellore	9	4131	8489	FREQUENT
12	Deepa Anand	Kochi	9	3785	8290	FREQUENT
4	Manoj Chandran	Chennai	8	2690	5465	FREQUENT

(149 rows returned; first 12 shown.)

Observation: Trips, distance and spend per passenger with a FREQUENT / REGULAR / OCCASIONAL classification.

5.9 Revenue by Train

```
-- Q9 REVENUE BY TRAIN  (route subtotals and grand total with ROLLUP)
--    The refund per ticket is worked out once in the inline view, then summed.
SELECT NVL(route_name, '** ALL ROUTES **')                                    AS
route_name,
       NVL(train_name, DECODE(GROUPING(route_name), 0, '-- route total --', ' ')) AS
train_name,
       SUM(amount)                                                            AS
collected,
       SUM(refunded)                                                          AS
refunded,
       SUM(amount) - SUM(refunded)                                            AS
net_revenue
FROM  (SELECT ro.route_name, t.train_name, pm.amount,
              NVL(cn.refund_amount, 0)
              + CASE WHEN pm.pay_status = 'REFUNDED' AND cn.cancel_id IS NULL THEN
pm.amount ELSE 0 END AS refunded
       FROM   payment pm
              JOIN reservation r        ON r.pnr = pm.pnr
              JOIN train_schedule s     ON s.schedule_id = r.schedule_id
              JOIN train t              ON t.train_no = s.train_no
              JOIN route ro             ON ro.route_id = t.route_id
              LEFT JOIN cancellation cn ON cn.pnr = r.pnr) x
GROUP  BY ROLLUP (x.route_name, x.train_name)
ORDER  BY GROUPING(x.route_name), x.route_name, GROUPING(x.train_name), net_revenue
DESC;
```

ROUTE_NAME	TRAIN_NAME	COLLECTED	REFUNDED	NET_REVENUE
Bengaluru - Ernakulam	Ernakulam Intercity	59954	3980.7	55973.3
Bengaluru - Ernakulam	-- route total --	59954	3980.7	55973.3
Chennai - Bengaluru	Chennai Shatabdi	69147	9202.8	59944.2
Chennai - Bengaluru	Brindavan Express	57360	3536.7	53823.3
Chennai - Bengaluru	Lalbagh Express	36291	1009.8	35281.2
Chennai - Bengaluru	-- route total --	162798	13749.3	149048.7
Chennai - Coimbatore	Kovai Express	62140	1632.6	60507.4
Chennai - Coimbatore	Nilgiri Express	55107	1351.8	53755.2
Chennai - Coimbatore	-- route total --	117247	2984.4	114262.6
Chennai - Madurai	Pandian Express	71897	2127	69770
Chennai - Madurai	Vaigai Express	60744	2718	58026
Chennai - Madurai	Madurai Tejas	38775	2609.1	36165.9
Chennai - Madurai	-- route total --	171416	7454.1	163961.9
ALL ROUTES	NULL	511415	28168.5	483246.5

Observation: Collected, refunded and net revenue per train with route subtotals and a grand total using GROUP BY ROLLUP. Net revenue across all routes is ₹4,83,246.50.

5.10 Routes Requiring Additional Capacity

```
-- Q10 ROUTES REQUIRING ADDITIONAL CAPACITY  (HAVING on aggregated journey data)
SELECT ro.route_name,
       COUNT(*)                                                  AS journeys,
       SUM(CASE WHEN v.occupancy_pct >= 95 THEN 1 ELSE 0 END)    AS
sold_out_journeys,
       SUM(v.waitlisted)                                         AS
unserved_passengers,
       ROUND(100 * SUM(v.seats_occupied) / SUM(v.capacity), 2)   AS occupancy_pct
FROM   v_reservation_performance v JOIN route ro ON ro.route_id = v.route_id
GROUP  BY ro.route_name
HAVING SUM(v.waitlisted) > 0
   AND 100 * SUM(v.seats_occupied) / SUM(v.capacity) >= 75
ORDER  BY unserved_passengers DESC;
```

ROUTE_NAME	JOURNEYS	SOLD_OUT_JOURNEYS	UNSERVED_PASSENGERS	OCCUPANCY_PCT
Chennai - Bengaluru	9	4	12	85
Chennai - Madurai	9	2	8	78.18

Observation: Routes with waitlisted (unserved) passengers and at least 75% overall occupancy, filtered with HAVING. Chennai–Bengaluru and Chennai–Madurai qualify.

6. Advanced SQL

6.1 Joins Involving Multiple Tables

A1: complete travel history of a passenger. Ten table references are joined along their foreign keys; STATION is joined twice (boarding and destination) with different aliases, and SEAT and COACH are outer-joined so that tickets without a seat are not lost.

```
-- A1 MULTI-TABLE JOIN (10 relations) : complete travel history of one passenger.
--    Station is joined twice (boarding and destination) using table aliases.
SELECT r.pnr, t.train_no, t.train_name, s.journey_date,
       fs.station_name AS boarding, ts.station_name AS destination,
       cc.class_name, c.coach_label || '-' || st.seat_no || ' ' || st.berth_type AS
seat,
       r.fare, pm.pay_mode, r.status
FROM   passenger p
       JOIN reservation r     ON r.passenger_id = p.passenger_id
       JOIN train_schedule s  ON s.schedule_id = r.schedule_id
       JOIN train t           ON t.train_no = s.train_no
       JOIN station fs        ON fs.station_code = r.from_station
       JOIN station ts        ON ts.station_code = r.to_station
       JOIN coach_class cc    ON cc.class_code = r.class_code
       JOIN payment pm        ON pm.pnr = r.pnr
       LEFT JOIN seat st      ON st.seat_id = r.seat_id
       LEFT JOIN coach c      ON c.coach_id = st.coach_id
WHERE  p.passenger_id = 1
ORDER  BY s.journey_date;
```

PNR	TRAIN_NAME	JOURNEY_DATE	BOARDING	DESTINATION	SEAT	FARE	STATUS
4200000015	Brindavan Express	20-SEP-2026	Chennai Central	KSR Bengaluru City	E1-4 AISLE	1101	COMPLETED
4200000189	Chennai Shatabdi	22-SEP-2026	Chennai Central	KSR Bengaluru City	C2-3 AISLE	561	COMPLETED
4200000309	Vaigai Express	23-SEP-2026	Chennai Central	Madurai Junction	C3-1 WINDOW	755	COMPLETED
4200000728	Ernakulam Intercity	28-SEP-2026	KSR Bengaluru City	Ernakulam Junction	C2-12 AISLE	893	COMPLETED
4200000132	Lalbagh Express	05-OCT-2026	Chennai Central	KSR Bengaluru City	C2-2 MIDDLE SEAT	561	COMPLETED
4200000081	Brindavan Express	21-OCT-2026	Chennai Central	KSR Bengaluru City	C2-12 AISLE	561	CONFIRMED
4200000371	Vaigai Express	24-OCT-2026	Chennai Central	Villupuram Junction	C3-4 WINDOW	271	CONFIRMED

6.2 Subqueries

6.2.1 Trains with occupancy above their route average (correlated subquery)

```
-- A2 CORRELATED SUBQUERY : trains whose occupancy is above the average occupancy of
THEIR route.
--    The inner query is re-evaluated for each outer row using u.route_id.
SELECT u.train_no, u.train_name, u.route_label, u.occupancy_pct,
       (SELECT ROUND(AVG(x.occupancy_pct), 2) FROM v_train_utilization x WHERE
x.route_id = u.route_id) AS route_avg_pct
FROM   v_train_utilization u
WHERE  u.occupancy_pct > (SELECT AVG(x.occupancy_pct)
                          FROM   v_train_utilization x
                          WHERE  x.route_id = u.route_id)
ORDER  BY u.route_id, u.occupancy_pct DESC;
```

TRAIN_NO	TRAIN_NAME	ROUTE_LABEL	OCCUPANCY_PCT	ROUTE_AVG_PCT
12027	Chennai Shatabdi	Chennai-Bengaluru	96.88	85.88
12639	Brindavan Express	Chennai-Bengaluru	96.88	85.88
12637	Pandian Express	Chennai-Madurai	96.83	76.07
12635	Vaigai Express	Chennai-Madurai	82.41	76.07
12675	Kovai Express	Chennai-Coimbatore	78.13	68.78

For each train the inner query averages the occupancy of trains on the same route (x.route_id = u.route_id), so it is re-evaluated per outer row.

6.2.2 Passengers with repeated cancellations (scalar subquery, LISTAGG, HAVING)

```
-- A3 PASSENGERS WITH REPEATED CANCELLATIONS (2 or more) with cancellation ratio
(scalar subquery)
SELECT p.passenger_id, p.full_name,
       COUNT(*)                                                          AS
cancellations,
       (SELECT COUNT(*) FROM reservation x WHERE x.passenger_id = p.passenger_id) AS
total_bookings,
       ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM reservation x WHERE x.passenger_id
= p.passenger_id), 1)
                                                                         AS
cancel_ratio_pct,
       LISTAGG(t.train_no, ', ') WITHIN GROUP (ORDER BY s.journey_date)   AS
trains_cancelled
FROM   reservation r
       JOIN passenger p      ON p.passenger_id = r.passenger_id
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
WHERE  r.status = 'CANCELLED'
GROUP  BY p.passenger_id, p.full_name
HAVING COUNT(*) >= 2
ORDER  BY cancellations DESC;
```

PASSENGER_ID	FULL_NAME	CANCELLATIONS	TOTAL_BOOKINGS	CANCEL_RATIO_PCT	TRAINS_CANCELLED
23	Nandhini Prasad	7	13	53.8	22671, 12677, 12027, 12637, 12671, 12027, 12635
15	Ajay Reddy	4	17	23.5	22671, 22671, 12675, 12027
7	Gayathri Varma	4	12	33.3	12027, 12635, 12607, 12027
42	Anand Velu	3	10	30	12027, 12671, 12677
31	Arjun Rao	3	11	27.3	12639, 22671, 12675

Cancelled tickets are grouped per passenger and kept when there are two or more; a correlated scalar subquery counts each passenger's total bookings for the ratio.

6.2.3 Routes with above-average passenger demand (nested subquery)

```
-- A4 NESTED SUBQUERY : routes whose passenger demand is above the average route
demand
SELECT ro.route_name, COUNT(*) AS passengers
FROM   reservation r
       JOIN train_schedule s ON s.schedule_id = r.schedule_id
       JOIN train t          ON t.train_no = s.train_no
       JOIN route ro         ON ro.route_id = t.route_id
WHERE  r.status <> 'CANCELLED'
GROUP  BY ro.route_name
```

```
HAVING COUNT(*) > (SELECT AVG(route_pax)
                   FROM  (SELECT COUNT(*) AS route_pax
                          FROM   reservation r2
                                 JOIN train_schedule s2 ON s2.schedule_id =
r2.schedule_id
                                 JOIN train t2          ON t2.train_no = s2.train_no
                          WHERE  r2.status <> 'CANCELLED'
                          GROUP  BY t2.route_id))
ORDER  BY passengers DESC;
```

ROUTE_NAME	PASSENGERS
Chennai - Bengaluru	267
Chennai - Madurai	266

The innermost query counts passengers per route, the middle query averages those counts (185.25), and HAVING keeps routes above the average.

6.2.4 Stations handling higher-than-average traffic (WITH clause)

```
-- A5 STATIONS HANDLING HIGHER-THAN-AVERAGE PASSENGER TRAFFIC (WITH clause + subquery)
WITH footfall AS (
  SELECT station_code, COUNT(*) AS movements
  FROM  (SELECT from_station AS station_code FROM reservation WHERE status IN
('CONFIRMED','COMPLETED')
         UNION ALL
         SELECT to_station FROM reservation WHERE status IN ('CONFIRMED','COMPLETED'))
  GROUP  BY station_code)
SELECT st.station_code, st.station_name, f.movements,
       ROUND((SELECT AVG(movements) FROM footfall), 1) AS network_avg
FROM   footfall f JOIN station st ON st.station_code = f.station_code
WHERE  f.movements > (SELECT AVG(movements) FROM footfall)
ORDER  BY f.movements DESC;
```

STATION_CODE	STATION_NAME	MOVEMENTS	NETWORK_AVG
MAS	Chennai Central	557	110.9
SBC	KSR Bengaluru City	271	110.9
MDU	Madurai Junction	226	110.9
CBE	Coimbatore Junction	145	110.9

A common table expression combines boardings and alightings with UNION ALL; stations above the network average (110.9 movements) are listed.

6.2.5 Trains requiring capacity enhancement (NOT EXISTS / EXISTS)

```
-- A6 TRAINS REQUIRING CAPACITY ENHANCEMENT (correlated NOT EXISTS + EXISTS):
--    every journey ran at >= 90% occupancy AND at least one passenger was left on the
waitlist
SELECT u.train_no, u.train_name, u.occupancy_pct, u.waitlisted
FROM   v_train_utilization u
WHERE  NOT EXISTS (SELECT 1 FROM v_reservation_performance v
                   WHERE  v.train_no = u.train_no AND v.occupancy_pct < 90)
AND    EXISTS     (SELECT 1 FROM reservation r JOIN train_schedule s ON s.schedule_id
= r.schedule_id
                   WHERE  s.train_no = u.train_no AND r.status = 'WAITLIST');
```

TRAIN_NO	TRAIN_NAME	OCCUPANCY_PCT	WAITLISTED
12639	Brindavan Express	96.88	7
12027	Chennai Shatabdi	96.88	5
12637	Pandian Express	96.83	8

"Every journey at least 90% full" is expressed as "no journey below 90%" (NOT EXISTS), combined with the existence of waitlisted passengers (EXISTS).

6.2.6 Passengers spending above their city's average (correlated subquery in HAVING)

```
-- A7 CORRELATED : passengers whose total spend exceeds the average spend of
passengers from the same city
SELECT p.passenger_id, p.full_name, p.home_city, SUM(r.fare) AS spent
FROM   passenger p JOIN reservation r ON r.passenger_id = p.passenger_id
WHERE  r.status IN ('CONFIRMED','COMPLETED')
GROUP  BY p.passenger_id, p.full_name, p.home_city
HAVING SUM(r.fare) > (SELECT SUM(r2.fare) / COUNT(DISTINCT r2.passenger_id)
                      FROM   reservation r2 JOIN passenger p2 ON p2.passenger_id =
r2.passenger_id
                      WHERE  r2.status IN ('CONFIRMED','COMPLETED')
                      AND    p2.home_city = p.home_city)
ORDER  BY p.home_city, spent DESC;
```

PASSENGER_ID	FULL_NAME	HOME_CITY	SPENT
13	Charulatha Sundar	Bengaluru	8358
6	Janani Shankar	Bengaluru	7515
18	Asha Hegde	Bengaluru	5670
105	Ramesh Chandran	Bengaluru	5582
63	Harini Nair	Bengaluru	4906
36	Harini Mani	Bengaluru	4177
35	Harini Hegde	Bengaluru	3918
125	Deepa Raghavan	Bengaluru	3643
86	Deepak Shankar	Bengaluru	3511
15	Ajay Reddy	Chennai	11039

(64 rows returned; first 10 shown.)

Each passenger's total spend is compared with the average spend per traveller from the same home city.

6.2.7 Loyal passengers (IN and NOT IN)

```
-- A8 IN / NOT IN : loyal passengers - 3 or more trips on near-full trains (>= 90%
occupancy), never cancelled
SELECT p.passenger_id, p.full_name, p.home_city
FROM   passenger p
WHERE  p.passenger_id IN (SELECT r.passenger_id
                          FROM   reservation r JOIN train_schedule s ON s.schedule_id
= r.schedule_id
                          WHERE  r.status IN ('CONFIRMED','COMPLETED')
                          AND    s.train_no IN (SELECT train_no FROM
v_train_utilization WHERE occupancy_pct >= 90)
                          GROUP  BY r.passenger_id
                          HAVING COUNT(*) >= 3)
AND    p.passenger_id NOT IN (SELECT passenger_id FROM reservation WHERE status =
'CANCELLED')
ORDER  BY p.passenger_id;
```

PASSENGER_ID	FULL_NAME	HOME_CITY
1	Balaji Mani	Kochi
3	Nithya Balaji	Chennai
6	Janani Shankar	Bengaluru
9	Fathima Elango	Chennai
10	Divya Arumugam	Madurai
12	Deepa Anand	Kochi
13	Charulatha Sundar	Bengaluru
18	Asha Hegde	Bengaluru
21	Shalini Raman	Chennai
32	Sanjay Balaji	Chennai

(32 rows returned; first 10 shown.)

Passengers with three or more trips on near-full trains who never cancelled. NOT IN is safe because passenger_id is NOT NULL.

6.2.8 Under-used coaches (correlated subquery on a CTE)

```
-- A9 CORRELATED (on a WITH clause) : coaches whose occupancy % is below the average
of their own train's coaches
WITH coach_occ AS (
  SELECT c.train_no, c.coach_id, c.coach_label, c.class_code,
         ROUND(100 * COUNT(r.pnr) / (cc.seats_per_coach * j.journeys), 1) AS
occupancy_pct
  FROM   coach c
         JOIN coach_class cc ON cc.class_code = c.class_code
         JOIN (SELECT train_no, COUNT(*) AS journeys FROM train_schedule GROUP BY
train_no) j ON j.train_no = c.train_no
         JOIN seat st ON st.coach_id = c.coach_id
         LEFT JOIN reservation r ON r.seat_id = st.seat_id AND r.status IN
('CONFIRMED','COMPLETED')
  GROUP  BY c.train_no, c.coach_id, c.coach_label, c.class_code, cc.seats_per_coach,
j.journeys)
SELECT o.train_no, o.coach_label, o.class_code, o.occupancy_pct,
       (SELECT ROUND(AVG(x.occupancy_pct), 1) FROM coach_occ x WHERE x.train_no =
o.train_no) AS train_avg_pct
FROM   coach_occ o
WHERE  o.occupancy_pct < (SELECT AVG(x.occupancy_pct) FROM coach_occ x WHERE
x.train_no = o.train_no) - 10
ORDER  BY o.train_no, o.coach_label;
```

TRAIN_NO	COACH_LABEL	CLASS_CODE	OCCUPANCY_PCT	TRAIN_AVG_PCT
12671	B1	3A	46.7	57.1
12671	H1	1A	41.7	57.1
12677	C2	CC	31.3	42
22671	E1	EC	33.3	47.2

Coaches whose occupancy is more than 10 points below the average of their own train's coaches.

6.3 Creation and Use of Views

Three views encapsulate the reporting logic so that reports read them instead of repeating the calculations.

View 1: v_reservation_performance (one row per journey)

```
-- VIEW 1 : RESERVATION PERFORMANCE - one row per journey (train + date)
CREATE OR REPLACE VIEW v_reservation_performance AS
SELECT s.schedule_id,
       t.train_no,
       t.train_name,
       t.route_id,
       s.journey_date,
       s.sched_status,
       cap.capacity,
       COUNT(r.pnr)                                                    AS
total_bookings,
       COUNT(CASE WHEN r.status IN ('CONFIRMED','COMPLETED') THEN 1 END) AS
seats_occupied,
       COUNT(CASE WHEN r.status = 'WAITLIST' THEN 1 END)               AS
waitlisted,
       COUNT(CASE WHEN r.status = 'CANCELLED' THEN 1 END)              AS cancelled,
```

```
       ROUND(100 * COUNT(CASE WHEN r.status IN ('CONFIRMED','COMPLETED') THEN 1 END) /
cap.capacity, 2)
                                                                              AS
occupancy_pct,
       ROUND(100 * COUNT(CASE WHEN r.status = 'CANCELLED' THEN 1 END) /
NULLIF(COUNT(r.pnr), 0), 2)
                                                                              AS
cancellation_rate,
       NVL(SUM(CASE WHEN r.status IN ('CONFIRMED','COMPLETED') THEN r.fare
                    WHEN r.status = 'CANCELLED' THEN r.fare - cn.refund_amount END),
0)
                                                                              AS revenue
FROM   train_schedule s
       JOIN train t ON t.train_no = s.train_no
       JOIN (SELECT c.train_no, SUM(cc.seats_per_coach) AS capacity
             FROM   coach c JOIN coach_class cc ON cc.class_code = c.class_code
             GROUP  BY c.train_no) cap ON cap.train_no = t.train_no
       LEFT JOIN reservation  r  ON r.schedule_id = s.schedule_id
       LEFT JOIN cancellation cn ON cn.pnr = r.pnr
GROUP  BY s.schedule_id, t.train_no, t.train_name, t.route_id, s.journey_date,
s.sched_status, cap.capacity;
```

SCHEDULE_ID	TRAIN_NO	JOURNEY_DATE	CAPACITY	TOTAL_BOOKINGS	SEATS_OCCUPIED	WAITLISTED	CANCELLED	OCCUPANCY_PCT	CANCELLATION_RATE	REVENUE
107	12027	22-SEP-2026	32	39	32	2	5	100	12.82	21269.6
108	12027	06-OCT-2026	32	38	31	2	5	96.88	13.16	19337.6
109	12027	23-OCT-2026	32	35	30	1	4	93.75	11.43	19110
104	12607	21-SEP-2026	36	25	24	0	1	66.67	4	12413.1
105	12607	05-OCT-2026	36	24	23	0	1	63.89	4.17	11755.1
106	12607	22-OCT-2026	36	22	22	0	0	61.11	0	11113
110	12635	23-SEP-2026	36	32	31	0	1	86.11	3.13	22906.5
111	12635	07-OCT-2026	36	32	30	0	2	83.33	6.25	19846
112	12635	24-OCT-2026	36	29	28	0	1	77.78	3.45	15273.5
113	12637	24-SEP-2026	42	45	42	3	0	100	0	24272

(28 rows returned; first 10 shown.)

View 2: v_train_utilization (one row per train)

```
-- VIEW 2 : TRAIN UTILIZATION - one row per train across all its journeys
CREATE OR REPLACE VIEW v_train_utilization AS
SELECT v.train_no,
       v.train_name,
       v.route_id,
       so.city || '-' || sd.city                                              AS
route_label,
       COUNT(*)                                                               AS journeys,
       SUM(v.capacity)                                                        AS
capacity_offered,
       SUM(v.seats_occupied)                                                  AS
seats_occupied,
       SUM(v.waitlisted)                                                      AS
waitlisted,
       SUM(v.cancelled)                                                       AS cancelled,
       SUM(v.total_bookings)                                                  AS
total_bookings,
       ROUND(100 * SUM(v.seats_occupied) / SUM(v.capacity), 2)                AS
occupancy_pct,
       ROUND(100 * SUM(v.cancelled) / NULLIF(SUM(v.total_bookings), 0), 2) AS
cancellation_rate,
       ROUND(100 * (SUM(v.seats_occupied) + SUM(v.waitlisted)) / SUM(v.capacity), 2)
```

```
AS demand_pct,
       SUM(v.revenue)                                                    AS revenue
FROM   v_reservation_performance v
       JOIN route   ro ON ro.route_id = v.route_id
       JOIN station so ON so.station_code = ro.origin_code
       JOIN station sd ON sd.station_code = ro.dest_code
GROUP  BY v.train_no, v.train_name, v.route_id, so.city, sd.city;
```

TRAIN_NO	TRAIN_NAME	ROUTE_LABEL	JOURNEYS	CAPACITY_OFFERED	SEATS_OCCUPIED	WAITLISTED	OCCUPANCY_PCT	CANCELLATION_RATE	DEMAND_PCT
12639	Brindavan Express	Chennai-Bengaluru	3	96	93	7	96.88	1.96	104.17
12027	Chennai Shatabdi	Chennai-Bengaluru	3	96	93	5	96.88	12.5	102.08
12637	Pandian Express	Chennai-Madurai	3	126	122	8	96.83	0.76	103.17
12635	Vaigai Express	Chennai-Madurai	3	108	89	0	82.41	4.3	82.41
12675	Kovai Express	Chennai-Coimbatore	3	96	75	0	78.13	3.85	78.13
12607	Lalbagh Express	Chennai-Bengaluru	3	108	69	0	63.89	2.82	63.89
12671	Nilgiri Express	Chennai-Coimbatore	3	138	82	0	59.42	3.53	59.42
22671	Madurai Tejas	Chennai-Madurai	3	96	47	0	48.96	7.84	48.96
12677	Ernakulam Intercity	Bengaluru-Kochi	4	128	51	0	39.84	5.56	39.84

View 3: v_ticket_details (readable ticket for enquiries)

```
-- VIEW 3 : TICKET DETAILS - de-normalised ticket for enquiry screens
CREATE OR REPLACE VIEW v_ticket_details AS
SELECT r.pnr, p.passenger_id, p.full_name, t.train_no, t.train_name, s.journey_date,
       r.from_station, r.to_station, r.class_code,
       c.coach_label, st.seat_no, st.berth_type,
       (rs2.km_from_origin - rs1.km_from_origin) AS distance_km,
       r.fare, r.status
FROM   reservation r
       JOIN passenger p       ON p.passenger_id = r.passenger_id
       JOIN train_schedule s  ON s.schedule_id = r.schedule_id
       JOIN train t           ON t.train_no = s.train_no
       JOIN route_stop rs1    ON rs1.route_id = t.route_id AND rs1.station_code =
r.from_station
       JOIN route_stop rs2    ON rs2.route_id = t.route_id AND rs2.station_code =
r.to_station
       LEFT JOIN seat st      ON st.seat_id = r.seat_id
       LEFT JOIN coach c      ON c.coach_id = st.coach_id;
```

6.4 Purpose of the Advanced Queries

Query	Technique	Business purpose
A1	10-table join, self-join of STATION, outer joins	Passenger enquiry: complete travel history
A2	Correlated scalar subquery	Compare each train with peers on the same route
A3	GROUP BY / HAVING, scalar subquery, LISTAGG	Identify passengers who cancel repeatedly
A4	Nested subquery in HAVING	Find corridors with above-average demand
A5	WITH clause, UNION ALL	Find the busiest stations for staffing and facilities
A6	Correlated NOT EXISTS + EXISTS	Trains that are consistently full and turn passengers away
A7	Correlated subquery in HAVING, COUNT(DISTINCT)	High-value customers per city
A8	IN / NOT IN with nested subqueries	Loyal customers for reward programmes
A9	CTE + correlated subquery	Coaches that could be removed or reassigned
Views	Aggregation, outer joins, NULLIF	Single definition of occupancy, demand and revenue

Query	Technique	Business purpose
		reused by all reports

6.5 BTL-6 Challenge: Train Capacity Risk Identification Module

The module classifies each train using three measures from `v_train_utilization`: **occupancy %** (seats occupied ÷ capacity), **cancellation rate %** (cancelled ÷ booked) and **average demand %** ((occupied + waitlisted) ÷ capacity, which exceeds 100% when passengers are turned away).

Risk status	Criteria	Justification and recommended action
CAPACITY REQUIRED	Occupancy ≥ 90% AND demand ≥ 100% AND cancellation rate ≤ 10%	The train is full, passengers are being turned away, and so few tickets are cancelled that the waitlist will not clear by itself. Action: attach extra coaches or run a special train.
HIGH DEMAND	Not the above, AND (occupancy ≥ 75% OR demand ≥ 90%)	Strong demand that is still serviceable, or high demand offset by many cancellations that free seats. Action: monitor weekly, extra quota on peak days.
NORMAL	Everything else	Demand is within capacity. Below 50% occupancy the coach composition should be reviewed.

The thresholds are stored in the table RISK_THRESHOLD, so management can tune them without changing code; the function `fn_capacity_risk` applies the rules, `prc_capacity_risk_report` stores a daily classification in CAPACITY_RISK_LOG, and a what-if analysis tests a changed threshold inside a transaction that is rolled back.

```
CREATE OR REPLACE FUNCTION fn_capacity_risk (
  p_occupancy NUMBER, p_cancel_rate NUMBER, p_demand NUMBER
) RETURN VARCHAR2 IS
  v_cr_occ NUMBER; v_cr_dem NUMBER; v_cr_can NUMBER; v_hd_occ NUMBER; v_hd_dem NUMBER;
BEGIN
  SELECT MAX(DECODE(param_code, 'CR_MIN_OCCUPANCY', param_value)),
         MAX(DECODE(param_code, 'CR_MIN_DEMAND',    param_value)),
         MAX(DECODE(param_code, 'CR_MAX_CANCEL',    param_value)),
         MAX(DECODE(param_code, 'HD_MIN_OCCUPANCY', param_value)),
         MAX(DECODE(param_code, 'HD_MIN_DEMAND',    param_value))
  INTO   v_cr_occ, v_cr_dem, v_cr_can, v_hd_occ, v_hd_dem
  FROM   risk_threshold;

  IF p_occupancy >= v_cr_occ AND p_demand >= v_cr_dem AND NVL(p_cancel_rate, 0) <=
v_cr_can THEN
    RETURN 'CAPACITY REQUIRED';
  ELSIF p_occupancy >= v_hd_occ OR p_demand >= v_hd_dem THEN
    RETURN 'HIGH DEMAND';
  END IF;
  RETURN 'NORMAL';
END;
/
```

```
-- 10.1 DECISION-SUPPORT REPORT
SELECT u.train_no                                                    AS "Train",
       u.route_label                                                 AS "Route",
       u.occupancy_pct                                               AS "Occupancy
%",
       u.cancellation_rate                                           AS
"Cancellation Rate",
       u.demand_pct                                                  AS "Average
Demand",
       ROUND(AVG(u.occupancy_pct) OVER (PARTITION BY u.route_id), 2)    AS "Route Avg
```

```
Occ %",
       fn_capacity_risk(u.occupancy_pct, u.cancellation_rate, u.demand_pct) AS "Risk
Status",
       CASE fn_capacity_risk(u.occupancy_pct, u.cancellation_rate, u.demand_pct)
            WHEN 'CAPACITY REQUIRED' THEN 'Add ' || CEIL(u.waitlisted / u.journeys) ||
' seat(s)/journey: attach extra coach'
            WHEN 'HIGH DEMAND'       THEN 'Monitor weekly; extra quota on peak days'
            ELSE CASE WHEN u.occupancy_pct < 50 THEN 'Under-used: review coach
composition'
                      ELSE 'No action' END
       END                                                              AS
"Recommended Action"
FROM   v_train_utilization u
ORDER  BY DECODE(fn_capacity_risk(u.occupancy_pct, u.cancellation_rate, u.demand_pct),
                 'CAPACITY REQUIRED', 1, 'HIGH DEMAND', 2, 3),
          u.occupancy_pct DESC;
```

Output: Train Capacity Risk Report

Train	Route	Occupancy %	Cancellation Rate	Average Demand	Risk Status	Recommended Action
12639	Chennai-Bengaluru	96.88	1.96	104.17	CAPACITY REQUIRED	Add 3 seat(s)/journey: attach extra coach
12637	Chennai-Madurai	96.83	0.76	103.17	CAPACITY REQUIRED	Add 3 seat(s)/journey: attach extra coach
12027	Chennai-Bengaluru	96.88	12.5	102.08	HIGH DEMAND	Monitor weekly; extra quota on peak days
12635	Chennai-Madurai	82.41	4.3	82.41	HIGH DEMAND	Monitor weekly; extra quota on peak days
12675	Chennai-Coimbatore	78.13	3.85	78.13	HIGH DEMAND	Monitor weekly; extra quota on peak days
12607	Chennai-Bengaluru	63.89	2.82	63.89	NORMAL	No action
12671	Chennai-Coimbatore	59.42	3.53	59.42	NORMAL	No action
22671	Chennai-Madurai	48.96	7.84	48.96	NORMAL	Under-used: review coach composition
12677	Bengaluru-Kochi	39.84	5.56	39.84	NORMAL	Under-used: review coach composition

Daily summary printed by prc_capacity_risk_report:

```
TRAIN CAPACITY RISK SUMMARY - 28-OCT-2026
CAPACITY REQUIRED   2 train(s): 12637, 12639
HIGH DEMAND         3 train(s): 12027, 12635, 12675
NORMAL              4 train(s): 12607, 12671, 12677, 22671
```

What-if analysis: raising the tolerated cancellation rate from 10% to 15% would move only train 12027 (Chennai Shatabdi, 12.5% cancellations) from HIGH DEMAND to CAPACITY REQUIRED. The change is rolled back after the analysis.

7. Results and Discussion

7.1 Summary of Results

Area	Result obtained
Data volume	14 stations, 4 routes, 21 stops, 9 trains, 30 coaches, 320 seats, 28 journeys, 150 passengers, 777 tickets, 777 payments, 36 cancellations
Integrity	All seven invalid operations were rejected; no seat is double-booked and every fare matches the formula
Booking demo	8 of 9 passengers confirmed on an 8-seat coach, 1 waitlisted; after one cancellation the waitlisted passenger was confirmed automatically with a 90% refund to the canceller
Network occupancy	Route occupancy ranges from 39.84% (Bengaluru–Ernakulam) to 85% (Chennai–Bengaluru)
Cancellations	Overall low (36 of 777 tickets); highest on Chennai Shatabdi (12.5%)
Revenue	Net revenue ₹4,83,246.50; Chennai–Madurai earns the most (₹1,63,961.90)
Capacity risk	2 trains CAPACITY REQUIRED (12639, 12637), 3 HIGH DEMAND (12027, 12635, 12675), 4 NORMAL

7.2 Discussion of Insights

- **Chennai–Bengaluru and Chennai–Madurai are under capacity pressure.** Both routes have waitlisted passengers and over 75% occupancy (Q10), and they have the highest demand ranks (Q6a, A4).
- **Brindavan Express (12639) and Pandian Express (12637) need extra coaches.** Every journey ran at 90% or more with passengers left on the waitlist (A6), and their low cancellation rates mean the waitlist does not clear. The report recommends about 3 additional seats per journey for each.
- **Chennai Shatabdi (12027) is full but volatile.** Its occupancy equals 12639's, but 12.5% of its tickets are cancelled, so freed seats absorb much of the waitlist; it is classified HIGH DEMAND rather than CAPACITY REQUIRED. The what-if analysis shows it is the first train to change category if the policy is relaxed.
- **Some services are under-used.** Ernakulam Intercity (39.84%) and Madurai Tejas (48.96%) run well below capacity; their coach composition should be reviewed (A9 also identifies individual under-used coaches).
- **Chennai Central dominates passenger traffic.** Over a third of all station movements are at MAS (Q2, A5), which is useful for staffing and platform planning.
- **Weekend travel peaks.** Sunday carries the largest share of passengers (21.9%, Q6c).
- **Database-level enforcement works.** Unique indexes, CHECK constraints and the trigger rejected every invalid operation, and the stored procedures keep seat allocation correct even under concurrent bookings.

7.3 Screenshots of SQL Query Execution and Outputs

Screenshots of the scripts executed in Oracle SQL Developer (Script Output, Query Result and DBMS Output tabs) are to be inserted here.

[Insert screenshot]

Figure 7.1: Execution of run_all.sql (Script Output)

[Insert screenshot]

Figure 7.2: Integrity test results (DBMS Output)

[Insert screenshot]

Figure 7.3: Booking and cancellation demo (DBMS Output)

[Insert screenshot]

Figure 7.4: Train-wise reservation statistics (Query Result)

[Insert screenshot]

Figure 7.5: Train Capacity Risk Report (Query Result)

8. Conclusion

This case study designed and implemented a complete relational database for railway reservation and passenger services. Requirement analysis produced ten functional requirements and sixteen business rules, which were modelled in an ER diagram with fourteen entities and converted into a BCNF relational schema with clearly defined primary, foreign and alternate keys.

The implementation in Oracle enforces data integrity at the database level: CHECK, UNIQUE and foreign-key constraints protect individual tables; function-based unique indexes guarantee that no seat is sold twice and that a passenger holds only one active ticket per journey; and a BEFORE row trigger validates the route direction and seat ownership and calculates every fare. Stored procedures implement booking with automatic seat allocation and waitlisting, and cancellation with time-based refunds and first-come-first-served waitlist promotion, both designed to remain correct under concurrent use.

Ten analytical queries, nine advanced queries and three views turn the operational data into management information on train performance, station traffic, route and coach utilisation, cancellations, revenue and passenger behaviour. The Train Capacity Risk Identification Module combines occupancy, cancellation rate and demand into justified, tunable rules, and identified Brindavan Express and Pandian Express as the trains that require additional capacity.

The solution replaces error-prone manual records with a consistent, non-redundant database that supports accurate seat allocation, reservation integrity and efficient retrieval of railway operational information. Possible extensions include segment-wise seat reuse, multi-passenger PNRs, concession-based fares and a web front end that calls the existing stored procedures.